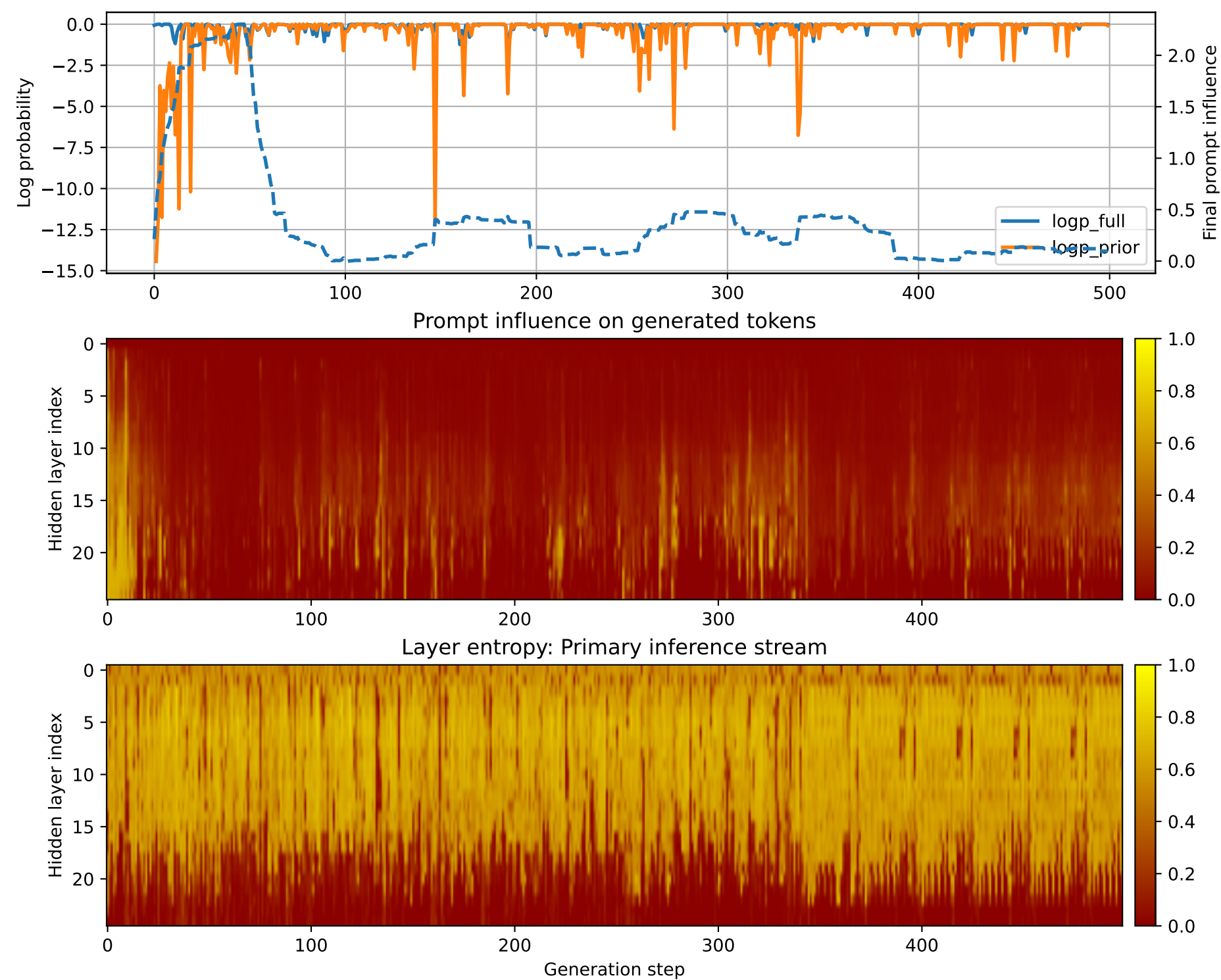


Prompt 1



PROMPT:
Task ID: PYC044 Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests. Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC044 Your code must contain this required symbol:
def binary_search Implement this task:
define function binary_search(xs,target) returning index or -1..
Start your answer with the check-token line,
then provide the Python implementation.

MODEL RESPONSE:
CHECK_TOKEN: PYC044 # Define function binary_search def
binary_search(xs, target): """ Returns the
index of the target value in the given list of integers, `xs`,
using a binary search algorithm. The binary
search algorithm works by repeatedly dividing the search interval
in half until the target value is found or
the search interval is empty. Args: xs (list[int]):
A list of integers to search through.
target (int): The value to find in the list. Returns:
int: The index of the target value in the
list, or -1 if the target value is not found. """ # Check
if the list is empty if len(xs) == 0:
return -1 # Initialize the left and right pointers left
= 0 right = len(xs) - 1 # Perform
binary search while left <= right: # Calculate the
middle index mid = (left + right) // 2
Check if the middle element is the target value if
xs[mid] == target: return mid
If the target value is less than the middle element, update the
left pointer to be the middle + 1
elif xs[mid] > target: right = mid - 1 # If
the target value is greater than the middle
element, update the right pointer to be the middle - 1
else: left = mid + 1 # If the
target value is not found, return -1 return -1 # Test the
binary_search function print(binary_search([1,
2, 3, 4, 5], 3)) # Output: 2 print(binary_search([1, 2, 3, 4,
5], 6)) # Output: -1 print(binary_search([1,
2, 3, 4, 5], 7)) # Output: -1 print(binary_search([1, 2, 3, 4,
5], 8)) # Output: -1 print(binary_search([1,
2, 3, 4, 5], 9)) # Output: -1 print(binary_search([1, 2, 3, 4,
5],